Assignment-13.5

Name : J.Akshaya

Ht.no : 2303A51629

Batch. No: 22

Task Description #1 (Refactoring – Removing Global Variables)

Prompt:

Refactor the given Python code to remove the global variable `rate`.

Pass the value as a function to improve modularity and testability and adding clear documentation.

Ensure the function works for different interest rates and take input from user.

rate = 0.1

def calculate_interest(amount):

    return amount * rate

print(calculate_interest(1000))

Code:

```
ass13.py > ...
# Refactor the given Python code to remove the global variable `rate`.
# Pass the value as a function to improve modularity and testability and adding clear documentation.
# Ensure the function works for different interest rates and take input from user. rate = 0.1
# def calculate_interest(amount): return amount * rate
# print(calculate_interest(1000))
def calculate_total_cost(amount, rate):
    total_cost=price*quantity*rate
    return total_cost
# Get user input for amount and interest rate
price= float(input("Enter the price: "))
rate = float(input("Enter the interest rate (as a decimal): "))
quantity = int(input("Enter the quantity: "))
# Calculate total cost using the function
total_cost = calculate_total_cost(price, rate)
print(f"The total cost is: {total_cost}")
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/ass13.py"
Enter the price: 2000
Enter the interest rate (as a decimal): 100
Enter the quantity: 2
The total cost is: 400000.0
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Observation:

The updated code gets rid of the global variable and instead passes the interest rate as a parameter, making it more modular. This way, the function is easier to test with different values and doesn't rely on outside variables. The structure gets simpler, easier to reuse, and easier to keep up over time.

Task Description #2 : (Refactoring Deeply Nested Conditionals)
Prompt:

Refactor the following Python code that contains deeply nested if–elif–else statements.

Simplify the control flow using guard clauses or a mapping-based approach to improve readability and maintainability.

Remove unnecessary nesting, apply clear variable naming, and improve the logic while keeping the same output format.

And take input from the user for the score.

Legacy Code:

```
score = 78
if score >= 90:
    print("Excellent")
else:
    if score >= 75:
        print("Very Good")
    else:
        if score >= 60:
            print("Good")
        else:
            print("Needs Improvement")
```

Code:

```python
# Refactor the following Python code that contains deeply nested if-elif-else statements.
# Simplify the control flow using guard clauses or a mapping-based approach to improve readability and maintainability.
# Remove unnecessary nesting, apply clear variable naming, and improve the logic while keeping the same output format.
# And take input from the user for the score. Legacy Code:


def evaluate_score(score):
    """
    Evaluates the score and returns a corresponding message.

    Parameters:
    score (int): The score to evaluate.

    Returns:
    str: A message corresponding to the score evaluation.
    """
    if score >= 90:
        return "Excellent"
    elif score >= 75:
        return "Very Good"
    elif score >= 60:
        return "Good"
    else:
        return "Needs Improvement"
# Get user input for the score
score = int(input("Enter the score: "))
# Evaluate the score and print the result
result = evaluate_score(score)
print(result)
```

Ln 17, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.13.12 (Microsof

Output:

```
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/ass13.py"
Enter the score: 95
Excellent
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/ass13.py"
Enter the score: 75
Very Good
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Observation:

The refactored code makes things easier to read by getting rid of those complicated nested ifs and using a simpler control structure instead.Simplifying the logic makes the program easier to follow and keep up, without changing what it actually does.This way, the code stays simpler and it's easier to keep up with any changes down the road.

Task Description #3 : (Refactoring Repeated File Handling Code)

Prompt:

Refactor the repeated file handling code using Python.

Apply DRY principle and context managers (with open()).

Create a reusable function that accepts the filename as a parameter, removes duplicate open/read/close code, improves variable naming,

and adds comments explaining the logic.Take input from the user for the filename.

Legacy Code:

```
f = open("data1.txt")

print(f.read())

f.close()

f = open("data2.txt")

print(f.read())

f.close()
```

```python
# Refactor the repeated file handling code using Python. Apply DRY principle and context managers (with open(
# Create a reusable function that accepts the filename as a parameter, removes duplicate open/read/close code
# and adds comments explaining the logic.Take input from the user for the filename.
# Legacy Code:
# f = open("data1.txt") print(f.read()) f.close()
# f = open("data2.txt") print(f.read()) f.close()

def read_file(filename):
    """
    Reads the content of a file and prints it.

    Parameters:
    filename (str): The name of the file to read.
    """
    try:
        with open(filename, 'r') as file:
            content = file.read()
            print(content)
    except FileNotFoundError:
        print(f"The file {filename} was not found.")
    except Exception as e:
        print(f"An error occurred: {e}")
# Get user input for the filename
filename = input("Enter the filename to read: ")
# Read and print the content of the specified file
read_file(filename)
print("File reading completed.")
```

Output:

```
ha/OneDrive/Desktop/assisantcoding - Copy/ass13.py"
Enter the filename to read: library
The file library was not found.
File reading completed.
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/aks
ha/OneDrive/Desktop/assisantcoding - Copy/ass13.py"
Enter the filename to read: data.txt
The file data.txt was not found.
File reading completed.
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/aks
ha/OneDrive/Desktop/assisantcoding - Copy/ass13.py"
Enter the filename to read: app.txt
The file app.txt was not found.
File reading completed.
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Observation:

The updated code gets rid of repeating file handling parts by adding a reusable function where you just pass in the file name.Using the with open() context manager helps manage resources better by making sure files get closed automatically when you're done with them.This method sticks to the DRY principle, makes the code easier to follow, and helps keep it simple to maintain.

Task Description #4 : (Optimizing Search Logic)

Prompt:

Refactor the given Python code that performs a linear search on a list of usernames.

Replace the list-based linear search with a more efficient data structure like a set or dictionary to achieve better time complexity.

Include meaningful variable names, comments explaining the changes, and mention the time complexity of the new approach

and Take input from the user for the username.

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

    if u == name:

        found = True

        break

print("Access Granted" if found else "Access Denied")

Code:

```
# Task Description #4 : (Optimizing Search Logic) Prompt:
# Refactor the given Python code that performs a linear search on a list of usernames.
# Replace the list-based linear search with a more efficient data structure like a set or dictionary to achieve better time co

# Include meaningful variable names, comments explaining the changes, and mention the time complexity of the new approach
# and Take input from the user for the username. users = ["admin", "guest", "editor", "viewer"] name = input("Enter username:
# found = False for u in users:
# if u == name: found = True break
# print("Access Granted" if found else "Access Denied")

def check_username(username):
    """
    Checks if the given username exists in the set of valid usernames.

    Parameters:
    username (str): The username to check.

    Returns:
    str: A message indicating whether access is granted or denied.
    """
    # Using a set for O(1) average time complexity for lookups
    valid_usernames = {"admin", "guest", "editor", "viewer"}

    if username in valid_usernames:
        return "Access Granted"
    else:
        return "Access Denied"
# Get user input for the username
username = input("Enter username: ")
print(check_username(username))
```

Output:

```
ha/OneDrive/Desktop/assisantcoding - Copy/ass13.py"
Enter username: admin
Access Granted
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/ak
ha/OneDrive/Desktop/assisantcoding - Copy/ass13.py"
Enter username: akshayaadmin
Access Denied
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/ak
ha/OneDrive/Desktop/assisantcoding - Copy/ass13.py"
Enter username: request
Access Denied
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Observation:

The updated program makes searching faster by switching from a list to a set, which cuts down the lookup time from O(n) to about O(1).The control flow is easier to follow by checking membership directly instead of using a loop with a flag variable.This makes the code cleaner, easier to read, and better in performance.

Task Description #5 : (Refactoring Procedural Code into OOP Design)
Prompt:

Refactor the given Python code into an Object-Oriented design.

Create a class named EmployeeSalaryCalculator that follows OOP principles and encapsulation.

Include attributes for salary and tax_rate, and methods to calculate tax and net salary.

Adding proper documentation. Ensure the code is clean, modular, and readabile. Take input from the user for salary and tax rate.

Legacy Code:

```
salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)
```

Code:

```python
class EmployeeSalaryCalculator:
    """A class to calculate the salary of employees based on their working hours and hourly rate.
    Attributes:
        name (str): The name of the employee.
        hours_worked (float): The number of hours worked by the employee.
        hourly_rate (float): The hourly rate of the employee.
        Methods:
        calculate_salary(): Calculates the total salary of the employee based on hours worked and hourly rate.
        calculate_net_salary(): Calculates the net salary of the employee after

    """
    def __init__(self,salary,tax_rate):
        self.salary = salary
        self.tax_rate = tax_rate
    def calculate_salary(self):
        return self.salary
    def calculate_net_salary(self):
        tax_amount = self.salary * self.tax_rate
        net_salary = self.salary - tax_amount
        return net_salary
# Example usage:
employee = EmployeeSalaryCalculator(5000, 0.2)
print("Gross Salary:", employee.calculate_salary())
print("Net Salary:", employee.calculate_net_salary())
```

Output:

```
Gross Salary: 5000
Net Salary: 4000.0
(base) PS C:\Users\WINDOWS\OneDrive
```

Observation:

The procedural code was changed into a class-based design by using object-oriented principles.Encapsulation was done by keeping salary and tax rate as attributes and using class methods to handle the calculations.The improved code is broken down into smaller parts, making it easier to reuse and simpler to keep up or add new features to.

Task Description #6 : (Refactoring for Performance Optimization)
Prompt:

Refactor the following Python code to improve performance and using built-in functions or mathematical formulas.

Rename variables meaningfully and improve the code structure. Compare execution time between the legacy loop implementation and the optimized version.

Take input from the user for the limit.

Legacy Code:

```
total = 0
for i in range(1, 1000000):
    if i % 2 == 0:
        total += i
print(total)
```

Code:

```python

# Refactor the following Python code to improve performance and using built-in functions or mathematical for
# Rename variables meaningfully and improve the code structure. Compare execution time between the legacy lo
# Take input from the user for the limit.

# Legacy Code:
# total = 0
# for i in range(1, 1000000): if i % 2 == 0:
# total += i print(total)


import time
def sum_of_even_numbers(limit):
    """
    Calculates the sum of even numbers up to a given limit using a mathematical formula.

    Parameters:
    limit (int): The upper limit for calculating the sum of even numbers.

    Returns:
    int: The sum of even numbers up to the given limit.
    """
    # Using the formula n(n + 1) where n is the number of even numbers up to the limit
    n = limit // 2
    return n * (n + 1)
# Get user input for the limit
limit = int(input("Enter the limit: "))
# Measure execution time for the optimized version
start_time = time.time()
optimized_sum = sum_of_even_numbers(limit)
end_time = time.time()
print(f"Sum of even numbers up to {limit}: {optimized_sum}")
print(f"Execution time for optimized version: {end_time - start_time:.6f} seconds")

```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/aksha/OneDrive/Deskto
p/assisantcoding - Copy/ass13.py"
Enter the limit: 10
Sum of even numbers up to 10: 30
Execution time for optimized version: 0.000010 seconds
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/aksha/OneDrive/Deskto
p/assisantcoding - Copy/ass13.py"
Enter the limit: 20
Sum of even numbers up to 20: 110
Execution time for optimized version: 0.000010 seconds
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Observation:

The refactored code boosts performance by swapping out the loop for a simple math formula and using built-in functions.This cuts down on the time it takes and gets rid of unneeded conditional checks inside the loop.Because of this, the improved version runs faster and gives the same results with code that's easier to read and maintain.

Task Description #7 : (Removing Hidden Side Effects)
Prompt:

Refactor the given Python code to remove hidden side effects caused by modifying a global mutable list.

Rewrite the function in a functional style so it returns a new list instead of mutating global state.

Add clear comments and Take input from the user.

Legacy Code:

```
data = []

def add_item(x):

    data.append(x)

add_item(10)

add_item(20)

print(data)
```

Code:

```python
# Refactor the given Python code to remove hidden side effects caused by modifying a global mutable list.
# Rewrite the function in a functional style so it returns a new list instead of mutating global state.
# Add clear comments and Take input from the user. Legacy Code:
# data = []
# def add_item(x): data.append(x)
# add_item(10) add_item(20) print(data)

# Code:

def add_item_to_list(item, data_list):
    """
    Adds an item to a new list based on the provided data list.

    Parameters:
    item: The item to be added to the list.
    data_list: The original list to which the item will be added.

    Returns:
    A new list containing all items from the original list plus the new item.
    """
    # Create a new list by combining the original list with the new item
    new_list = data_list + [item]
    return new_list
# Get user input for the item to add
item = input("Enter an item to add to the list: ")
# Initialize an empty list
data = []
# Add items to the list using the function and print the results
data = add_item_to_list(item, data)

print(data)
```

Output:

```
p/assisantcoding - Copy/ass13.py"
Enter an item to add to the list: 17
['17']
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/aksha/OneDriv
p/assisantcoding - Copy/ass13.py"
Enter an item to add to the list: 5
['5']
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/aksha/OneDriv
p/assisantcoding - Copy/ass13.py"
Enter an item to add to the list: 10
['10']
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Observation:

The rewritten code gets rid of hidden side effects by not changing the global list.The function now works in a functional programming style by creating and returning a new list instead of changing shared mutable state.This makes the code more predictable and easier to maintain, and it also simplifies testing.

Task Description #8 : (Refactoring Complex Input Validation Logic)

Prompt:

Refactor the following Python password validation code to improve readability.

Create separate functions: check_length(password), check_digit(password), and check_uppercase(password) that return True/False.

Implement a validate_password(password) function that calls these functions and prints appropriate error messages if a rule fails.

Add comments and Take input from the user.

Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
    if any(c.isdigit() for c in password):
        if any(c.isupper() for c in password):
            print("Valid Password")
        else:
            print("Must contain uppercase")
    else:
        print("Must contain digit")
else:
print("Password too short")
```

Code:

```python
# Refactor the following Python password validation code to improve readability.
# Create separate functions: check_length(password), check_digit(password), and check_uppercase(password) that return True/False.
# Implement a validate_password(password) function that calls these functions and prints appropriate error messages if a rule fails.
# Add comments and Take input from the user. Legacy Code:
# password = input("Enter password: ") if len(password) >= 8:
# if any(c.isdigit() for c in password):
# if any(c.isupper() for c in password): print("Valid Password")
# else:
# print("Must contain uppercase") else:
# print("Must contain digit") else:
# print("Password too short")


# Code:

def check_length(password):

    return len(password) >= 8
def check_digit(password):
    return any(c.isdigit() for c in password)

def check_uppercase(password):
    return any(c.isupper() for c in password)

def validate_password(password):

    if not check_length(password):
        print("Password too short")
        return False
    if not check_digit(password):
        print("Must contain digit")
        return False
    if not check_uppercase(password):
        print("Must contain uppercase")
        return False
    print("Valid Password")
    return True

# Get user input for the password
password = input("Enter password: ")
# Validate the password
validate_password(password)
```

Output:

```
p/assisantcoding - Copy/ass13.py"
Enter password: akshaya
Password too short
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/aksha/OneDrive/Deskto
p/assisantcoding - Copy/ass13.py"
Enter password: akshaya@14
Must contain uppercase
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/aksha/OneDrive/Deskto
p/assisantcoding - Copy/ass13.py"
Enter password: Akshaya@14
Valid Password
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Observation:

The code is easier to read. Each validation rule is in its function. This makes the program simple to test and debug and maintain. It is better, than using a lot of conditional statements. The code is easy to understand. We can reuse the validation logic. The validation logic is still the same as it was before. The code is easier to understand and reuse. The refactored code and the original code have the validation logic but the refactored code is simpler.